Top 100 Angular Interview Questions

For Experienced Frontend Developers

Angular Fundamentals & Architecture

1. What is Angular, and how does it differ from AngularJS?
2. Explain the Angular component lifecycle hooks and their order of execution.
3. What is the difference between a Component, Directive, and Pipe?
4. Explain modules (NgModule) and how standalone components change this in modern Angular.
5. What is the Angular compiler, and what is the difference between JIT and AOT compilation?
6. Explain data binding types: interpolation, property binding, event binding, and two-way binding.
7. What is the difference between ViewChild, ContentChild, ViewChildren, and ContentChildren?
8. Explain the differences between structural directives and attribute directives.
9. What is the purpose of ng-content and content projection?
10. Explain the difference between template-driven forms and reactive forms.
11. What is Angular's change detection mechanism, and how does Zone.js fit in?
12. Explain OnPush change detection strategy and when to use it.
13. What are signals in Angular, and how do they differ from observables for reactivity?
14. Explain the difference between *ngIf/*ngFor and the new control flow syntax (@if, @for, @switch).
15. What is the difference between ng-template, ng-container, and ng-content?

Dependency Injection & Services

1. Explain Angular's dependency injection system and the injector hierarchy.
2. What is the difference between providedIn: 'root', 'platform', and component-level providers?
3. Explain the differences between useClass, useValue, useFactory, and useExisting providers.
4. What is a singleton service, and how does Angular ensure a service is a singleton?
5. Explain hierarchical dependency injection and how child injectors can override parent providers.
6. What is the injection token, and when would you use InjectionToken?
7. Explain the inject() function and how it differs from constructor injection.
8. How do you share data between unrelated components using services?
9. What is the difference between a service and a factory in Angular's DI system?
10. Explain how Angular resolves circular dependency issues between services.

RxJS & Observables

1. What is the difference between Observables and Promises?
2. Explain the difference between switchMap, mergeMap, concatMap, and exhaustMap.
3. What is the difference between Subject, BehaviorSubject, ReplaySubject, and AsyncSubject?

4. Explain hot vs cold observables with examples.

5. What is the purpose of the async pipe, and why is it preferred over manual subscriptions?

6. Explain common RxJS operators: map, filter, tap, debounceTime, distinctUntilChanged.

7. How do you handle errors in RxJS streams using catchError and retry?

8. What is memory leak risk with subscriptions, and how do you prevent it (takeUntil, async pipe, DestroyRef)?

9. Explain the difference between combineLatest, forkJoin, zip, and merge.

10. What is a multicast observable, and how does share() or shareReplay() work?

Routing & Navigation

1. Explain Angular Router and how route configuration works.

2. What is the difference between a route guard's CanActivate, CanDeactivate, CanLoad, and CanMatch?

3. Explain lazy loading of modules/routes and its performance benefits.

4. What is the difference between RouterLink, navigate(), and navigateByUrl()?

5. Explain route resolvers and when you'd use them.

6. What are route parameters vs query parameters, and how do you access each?

7. Explain nested (child) routes and named outlets (auxiliary routes).

8. How does Angular handle route reuse strategy, and when would you customize it?

9. What is preloading strategy in Angular routing, and what options are available?

10. How do you implement role-based access control using route guards?

Forms & Validation

1. Explain the difference between FormControl, FormGroup, and FormArray.

2. How do you create a custom validator in Angular reactive forms?

3. What is a cross-field validator, and how do you implement one?

4. Explain how to create a custom form control using ControlValueAccessor.

5. What is the difference between synchronous and asynchronous validators?

6. How do you dynamically add or remove form controls at runtime?

7. Explain valueChanges and statusChanges observables on form controls.

8. What is the difference between markAsTouched, markAsDirty, and updateValueAndValidity?

9. How do you handle nested forms and form groups for complex objects?

10. Explain how form state (pristine, dirty, touched, valid) is used in UI feedback.

Performance Optimization

1. What techniques would you use to optimize the performance of a large Angular application?

2. Explain trackBy in *ngFor and why it improves rendering performance.

3. How does lazy loading modules improve initial load time?

4. What is tree-shaking, and how does Angular CLI leverage it during build?

5. Explain differential loading and how Angular serves different bundles to different browsers.

6. What is the impact of using pure pipes vs impure pipes on performance?

7. How do you reduce bundle size in an Angular application?

8. Explain virtual scrolling (CDK virtual scroll) and when to use it.

9. What is the role of Web Workers in Angular, and when would you use them?

10. How do you detect and fix memory leaks caused by detached DOM nodes or subscriptions?

Testing & Debugging

1. Explain the difference between TestBed, ComponentFixture, and DebugElement in Angular testing.

2. How do you mock a service dependency in a unit test using Jasmine/Jest?

3. What is the difference between shallow rendering and deep rendering in component tests?

4. Explain how to test components with asynchronous operations using fakeAsync and tick().

5. What is the difference between unit testing, integration testing, and end-to-end testing (Cypress/Playwright) in Angular?

6. How do you test a component that uses RxJS observables or HTTP calls?

7. Explain how to use spies to verify method calls in Jasmine tests.

8. What is the role of Angular's HttpTestingController in testing HTTP interactions?

9. How do you test route guards and resolvers?

10. What are common strategies for debugging change detection issues in Angular?

Advanced Concepts & Best Practices

1. Explain the difference between Angular Elements and standalone components.

2. What is server-side rendering (SSR) with Angular Universal, and what problems does it solve?

3. Explain state management approaches in Angular (services with RxJS, NgRx, Akita, signals-based stores).

4. What is the NgRx store, and how do actions, reducers, selectors, and effects interact?

5. How do you implement internationalization (i18n) in an Angular application?

6. Explain Angular's HttpClient interceptors and common use cases (auth tokens, error handling, logging).

7. What is the difference between a smart (container) component and a dumb (presentational) component?

8. Explain Angular's micro-frontend approaches (Module Federation with Angular).

9. How do you handle global error handling in Angular using ErrorHandler?

10. What is the difference between Angular CDK and Angular Material?

11. Explain how Angular's hydration works for SSR applications.

12. How do you secure an Angular application against XSS, and how does Angular's built-in sanitization help?

13. What is the difference between encapsulation modes (Emulated, ShadowDom, None) for component styles?

14. Explain how Angular handles environment-specific configuration (environment.ts files).

15. What are Angular schematics, and how do they help with code generation and migrations?